GUÍA DE REFERENCIA

Sentencias del lenguaje Python

Sintaxis, descripción y ejemplos ejecutados de todas las sentencias del lenguaje

Contenido

Conceptos fundamentales · Sentencias simples · Control de flujo · Excepciones y recursos · Definiciones · Ejemplo integrador · Tabla resumen

Basado en Python 3.12 · Octubre de 2026

Índice

1. Conceptos fundamentales 4

1.1 ¿Qué es una sentencia? 4

1.2 Sentencias simples y compuestas 4

1.3 Indentación, líneas lógicas y comentarios 5

1.4 Palabras reservadas 5

2. Sentencias simples 7

2.1 Sentencia de expresión 7

2.2 Asignación 7

2.3 Asignación aumentada 8

2.4 Asignación anotada 9

2.5 Expresión de asignación (operador morsa) 10

2.6 pass 11

2.7 del 11

2.8 return 12

2.9 yield 13

2.10 raise 14

2.11 break 15

2.12 continue 16

2.13 import 17

2.14 from ... import 18

2.15 global 18

2.16 nonlocal 19

2.17 assert 19

2.18 type 20

3. Sentencias de control de flujo 21

3.1 if / elif / else 21

3.2 while 22

3.3 for 23

3.4 match / case 25

4. Manejo de excepciones y recursos 27

4.1 try / except / else / finally 27

4.2 except* (grupos de excepciones) 29

4.3 with 30

5. Sentencias de definición 32

5.1 def (definición de funciones) 32

5.2 class (definición de clases) 33

5.3 Decoradores (@) 36

5.4 async def / await / async for / async with 37

6. Construcciones que parecen sentencias 39

6.1 lambda, comprensiones y expresión condicional 39

7. Ejemplo integrador 40

8. Tabla resumen de sentencias 42

Cómo leer esta guía. Cada sentencia se presenta con su **sintaxis**, una **descripción**, los **puntos clave** que conviene recordar y varios **ejemplos**. El código aparece sobre fondo claro y la **salida** sobre fondo oscuro. Todos los ejemplos fueron ejecutados con Python 3.12 al generar este documento, por lo que las salidas mostradas son reales.

CAPÍTULO 1

1. Conceptos fundamentales

Antes de revisar cada sentencia en detalle, conviene entender qué es una sentencia, en qué se diferencia de una expresión y cómo Python organiza el código en líneas y bloques.

1.1 ¿Qué es una sentencia?

Una **sentencia** (*statement*) es una instrucción completa que el intérprete de Python puede ejecutar. Un programa no es más que una secuencia de sentencias. Una **expresión**, en cambio, es un fragmento de código que *produce un valor* (`3 + 4` , `len(lista)` , `nombre.upper()`).

PUNTOS CLAVE

- Una sentencia **realiza una acción**: asignar, repetir, importar, definir, etc.
- Toda expresión puede usarse como sentencia (*sentencia de expresión*), pero no toda sentencia es una expresión.
- Las sentencias no devuelven valor: no se puede escribir `x = (y = 5)` .
- Python clasifica las sentencias en **simples** y **compuestas**.

EJEMPLOS

Ejemplo 1: Sentencias y expresiones

```
x = 3 + 4          # sentencia de asignación; "3 + 4" es una expresión
print(x)           # sentencia de expresión (una llamada a función)
if x > 5:          # sentencia compuesta (contiene un bloque)
    print("x es mayor que 5")
```

SALIDA

```
7
x es mayor que 5
```

1.2 Sentencias simples y compuestas

Las **sentencias simples** caben en una sola línea lógica y no contienen otras sentencias (`pass` , `return` , asignaciones, `import` ...). Las **sentencias compuestas** agrupan otras sentencias dentro de un bloque: tienen una *cabecera* que termina en dos puntos `:` seguida de una *suite* indentada.

PUNTOS CLAVE

- Simples: expresión, asignación, `pass` , `del` , `return` , `yield` , `raise` , `break` , `continue` , `import` , `global` , `nonlocal` , `assert` , `type` .
- Compuestas: `if` , `while` , `for` , `try` , `with` , `match` , `def` , `class` , `async def` , `async for` , `async with` .
- Varias sentencias simples pueden ir en la misma línea separadas por `;` (poco recomendable).
- Una suite corta puede ir en la misma línea que la cabecera, pero se recomienda usar siempre bloque indentado.

EJEMPLOS

Ejemplo 1: Suite en bloque y en una línea

```
n = 7
if n % 2 == 1:
    print("impar")      # suite indentada (forma recomendada)

if n > 5: print("mayor que 5")   # suite en la misma línea

a = 1; b = 2; c = a + b          # varias sentencias simples
print(a, b, c)
```

SALIDA

```
impar
mayor que 5
1 2 3
```

1.3 Indentación, líneas lógicas y comentarios

Python delimita los bloques mediante **indentación** (la convención PEP 8 es de 4 espacios por nivel), no con llaves. Una *línea lógica* puede extenderse en varias líneas físicas usando paréntesis, corchetes, llaves o una barra invertida `\` al final de la línea.

PUNTOS CLAVE

- Todas las sentencias de un mismo bloque deben tener la misma indentación.
- No mezcle tabuladores y espacios: Python 3 lo considera un error (`TabError`).
- Dentro de `()`, `[]` y `{}` los saltos de línea son libres.
- Los comentarios comienzan con `#` y llegan hasta el final de la línea.

EJEMPLOS

Ejemplo 1: Continuación de línea

```
total = (1 + 2 +
         3 + 4)                 # continuación implícita con paréntesis
texto = "Hola, " \
        "mundo"                 # continuación explícita con barra invertida
colores = [
    "rojo",
    "verde",     # los saltos de línea dentro de [] son libres
    "azul",
]
print(total, texto, colores)
```

SALIDA

```
10 Hola, mundo ['rojo', 'verde', 'azul']
```

1.4 Palabras reservadas

Las sentencias de Python se construyen con **palabras reservadas** (*keywords*) que no pueden usarse como nombres de variables. Puede consultarlas con el módulo `keyword`. Además existen las *soft keywords* (`match`, `case`, `type`, `_`), que sólo son reservadas en contextos específicos.

PUNTOS CLAVE

- Las palabras reservadas distinguen mayúsculas de minúsculas: `True`, `False` y `None` se escriben con inicial mayúscula.
- Intentar usar una palabra reservada como identificador produce un `SyntaxError`.

EJEMPLOS

Ejemplo 1: Listar las palabras reservadas

```
import keyword, textwrap

print("Reservadas:", len(keyword.kwlist))
print(textwrap.fill(" ".join(keyword.kwlist), 68))
print()
print("Soft keywords:", keyword.softkwlist)
```

SALIDA

```
Reservadas: 35
False None True and as assert async await break class continue def
del elif else except finally for from global if import in is lambda
nonlocal not or pass raise return try while with yield

Soft keywords: ['_', 'case', 'match', 'type']
```

CAPÍTULO 2

2. Sentencias simples

Las sentencias simples ocupan una única línea lógica. Son los bloques básicos con los que se construyen programas más complejos.

2.1 Sentencia de expresión

Cualquier expresión puede escribirse sola como sentencia. Se evalúa y su resultado se descarta (salvo en el modo interactivo, donde se muestra). Se usa sobre todo para **llamar a funciones y métodos** por sus efectos secundarios.

SINTAXIS

```
expresión
```

PUNTOS CLAVE

- Es la sentencia más común: `print(...)`, `lista.append(...)`, `archivo.close()`.
- Una expresión sin efecto (por ejemplo `3 + 4` sola) es válida pero inútil en un script.
- Las cadenas literales solas se usan como *docstrings* y comentarios de varias líneas.

EJEMPLOS

Ejemplo 1: Llamadas como sentencias

```
frutas = ["manzana", "pera"]
frutas.append("uva")          # sentencia de expresión: método con efecto
frutas.sort(reverse=True)     # modifica la lista, devuelve None
print(frutas)                 # llamada a función
len(frutas)                   # válida, pero su resultado se descarta
```

SALIDA

```
['uva', 'pera', 'manzana']
```

2.2 Asignación

La sentencia de asignación **vincula un nombre a un objeto** (Python no copia valores: enlaza referencias). El objetivo puede ser una variable, un atributo (`obj.x`), un índice (`lista[0]`), una porción (`lista[1:3]`) o una combinación de ellos mediante desempaquetado.

SINTAXIS

```
objetivo = expresión
objetivo1 = objetivo2 = expresión
a, b, *resto = iterable
```

PUNTOS CLAVE

- La expresión de la derecha se evalúa **primero**, completa, y luego se asigna.
- La asignación encadenada (`x = y = 0`) hace que todos los nombres apunten al mismo objeto.
- El desempaquetado funciona con cualquier iterable; `*resto` captura los elementos sobrantes en una lista.
- El intercambio `a, b = b, a` no requiere variable temporal.

EJEMPLOS

Ejemplo 1: Asignación simple, encadenada y múltiple

```
nombre = "Ana"
edad = 30
x = y = z = 0            # los tres nombres valen 0
a, b = 10, 20            # asignación múltiple (tupla implícita)
print(nombre, edad, x, y, z, a, b)
```

SALIDA

```
Ana 30 0 0 0 10 20
```

Ejemplo 2: Intercambio de valores

```
a, b = 1, 2
a, b = b, a
print(a, b)
```

SALIDA

```
2 1
```

Ejemplo 3: Desempaquetado extendido y anidado

```
primero, *medio, ultimo = [1, 2, 3, 4, 5]
print(primero, medio, ultimo)

(x, y), z = (1, 2), 3          # desempaquetado anidado
print(x, y, z)

letra1, letra2, letra3 = "abc"  # cualquier iterable
print(letra1, letra2, letra3)
```

SALIDA

```
1 [2, 3, 4] 5
1 2 3
a b c
```

Ejemplo 4: Asignar a índices, porciones, claves y atributos

```
lista = [1, 2, 3, 4, 5]
lista[0] = 99                # índice
lista[1:3] = [20, 30, 40]    # porción (puede cambiar la longitud)

persona = {}
persona["nombre"] = "Luis"   # clave de diccionario

class Punto:
    pass

p = Punto()
p.x = 5                      # atributo
print(lista, persona, p.x)
```

SALIDA

```
[99, 20, 30, 40, 4, 5] {'nombre': 'Luis'} 5
```

2.3 Asignación aumentada

Combina una operación con la asignación. `x += 1` equivale (aproximadamente) a `x = x + 1`, pero el objetivo se evalúa una sola vez y, en objetos **mutables**, la operación se realiza *in situ*.

SINTAXIS

```
objetivo OP= expresión      # OP: + - * / // % ** @ & | ^ >> <<
```

PUNTOS CLAVE

- Operadores disponibles: `+= -= *= /= //= %= **= @= &= |= ^= >>= <<=` .
- En listas, `+=` modifica el mismo objeto (como `extend`); `a = a + b` crea una lista nueva.
- En tipos inmutables (`int` , `str` , `tuple`) siempre se crea un objeto nuevo.

EJEMPLOS

Ejemplo 1: Operadores aumentados paso a paso

```
n = 10
n += 5;  print("n += 5   ->", n)
n -= 3;  print("n -= 3   ->", n)
n *= 2;  print("n *= 2   ->", n)
n //= 5; print("n //= 5  ->", n)
n **= 2; print("n **= 2  ->", n)
n %= 7;  print("n %= 7   ->", n)

texto = "Hola"
texto += " mundo"
print(texto)
```

SALIDA

```
n += 5   -> 15
n -= 3   -> 12
n *= 2   -> 24
n //= 5  -> 4
n **= 2  -> 16
n %= 7   -> 2
Hola mundo
```

Ejemplo 2: Diferencia con objetos mutables

```
a = [1, 2]
b = a                # b referencia la misma lista
a += [3]             # modifica la lista original in situ
print(a, b, a is b)

a = a + [4]          # crea una lista nueva
print(a, b, a is b)
```

SALIDA

```
[1, 2, 3] [1, 2, 3] True
[1, 2, 3, 4] [1, 2, 3] False
```

2.4 Asignación anotada

Permite **anotar el tipo** esperado de una variable (*type hints*). Las anotaciones son informativas: Python **no** las verifica en ejecución, pero herramientas como `mypy` , `pyright` y los editores sí las usan.

SINTAXIS

```
variable: tipo
variable: tipo = expresión
```

PUNTOS CLAVE

- Se puede anotar sin asignar valor (`nombre: str`); en ese caso la variable no queda definida.
- Las anotaciones de variables a nivel de módulo o clase se guardan en `__annotations__` .

- Puede anotarse cualquier tipo: `list[int]`, `dict[str, float]`, `int | None`, etc.

EJEMPLOS

Ejemplo 1: Anotaciones de tipo

```
edad: int = 25
nombre: str                         # sólo anotación, sin valor
precio: float = 9.99
etiquetas: list[str] = ["a", "b"]
opcional: int | None = None

print(edad, precio, etiquetas, opcional)
print(__annotations__["edad"], __annotations__["nombre"])

edad = "no es un entero"          # Python NO lo impide en ejecución
print(edad)
```

SALIDA

```
25 9.99 ['a', 'b'] None
<class 'int'> <class 'str'>
no es un entero
```

2.5 Expresión de asignación (operador morsa)

Introducido en Python 3.8. Asigna y **devuelve** el valor en una sola expresión. Técnicamente es una *expresión*, no una sentencia, pero se usa dentro de `if`, `while` y comprensiones para evitar cálculos repetidos.

SINTAXIS

```
(nombre := expresión)
```

PUNTOS CLAVE

- Casi siempre debe ir entre paréntesis.
- Útil para reutilizar un resultado en una condición y en el cuerpo.
- Evite abusar: puede reducir la legibilidad.

EJEMPLOS

Ejemplo 1: Uso en if y en comprensiones

```
datos = [1, 2, 3, 4, 5, 6]
if (n := len(datos)) > 5:
    print(f"Lista larga con {n} elementos")

valores = [3, 8, 12, 5]
grandes = [c for v in valores if (c := v * v) > 20]
print(grandes)
```

SALIDA

```
Lista larga con 6 elementos
[64, 144, 25]
```

Ejemplo 2: Uso en un while

```
entradas = iter(["uno", "dos", "", "tres"])
while (linea := next(entradas)) != "":
    print("Leído:", linea)
print("Fin: línea vacía")
```

SALIDA

```
Leído: uno
Leído: dos
Fin: línea vacía
```

2.6 pass

Sentencia **nula**: no hace nada. Se necesita cuando la sintaxis exige una sentencia pero no se desea ninguna acción (bloques vacíos, marcadores de posición).

SINTAXIS

```
pass
```

PUNTOS CLAVE

- Sirve para esbozar funciones, clases o ramas que se implementarán después.
- Es distinta de un comentario: un comentario no cuenta como sentencia, por lo que un bloque sólo con comentarios es un error.
- En una clase vacía de excepción es habitual: `class MiError(Exception): pass`.

EJEMPLOS

Ejemplo 1: Bloques vacíos

```
def por_implementar():
    pass                  # se completará más adelante

class Vacia:
    pass

class MiError(Exception):
    pass

for i in range(3):
    pass                  # bucle que no hace nada

if True:
    pass
print("El código con pass es válido")
```

SALIDA

```
El código con pass es válido
```

2.7 del

Elimina **referencias**: nombres, elementos de una secuencia, claves de un diccionario o atributos. El objeto en memoria se libera cuando ya no queda ninguna referencia a él.

SINTAXIS

```
del objetivo[, objetivo, ...]
```

PUNTOS CLAVE

- `del x` elimina el nombre `x` del espacio de nombres (no el objeto necesariamente).
- `del lista[i]` y `del lista[a:b]` eliminan elementos o porciones.

- `del dic[clave]` elimina una entrada (lanza `KeyError` si no existe).
- `del obj.atributo` elimina un atributo de instancia.

EJEMPLOS

Ejemplo 1: Eliminar nombres

```
x = 10
del x
try:
    print(x)
except NameError as e:
    print("Error:", e)
```

SALIDA

```
Error: name 'x' is not defined
```

Ejemplo 2: Eliminar elementos, porciones y claves

```
lista = [10, 20, 30, 40, 50]
del lista[0]          # elimina el índice 0
del lista[1:3]        # elimina una porción
print(lista)

d = {"a": 1, "b": 2, "c": 3}
del d["a"]
print(d)

class Caja:
    def __init__(self):
        self.contenido = "libros"

c = Caja()
del c.contenido
print(hasattr(c, "contenido"))
```

SALIDA

```
[20, 50]
{'b': 2, 'c': 3}
False
```

2.8 return

Termina la ejecución de una función y **devuelve** un valor a quien la llamó. Si no hay expresión (o si la función termina sin `return`), devuelve `None`.

SINTAXIS

```
return [expresión]
```

PUNTOS CLAVE

- Sólo puede usarse dentro de una función.
- Para devolver varios valores, devuelva una tupla: `return a, b`.
- Permite *salidas tempranas* (guard clauses) que simplifican el código.
- Dentro de un `try` con `finally`, el bloque `finally` se ejecuta antes de que la función retorne.

EJEMPLOS

Ejemplo 1: Retornos simples, múltiples y None

```
def dividir(a, b):
    if b == 0:
        return None            # salida temprana
    return a / b

def min_max(nums):
    return min(nums), max(nums)    # devuelve una tupla

def saludar(nombre):
    print("Hola", nombre)          # sin return -> None

print(dividir(10, 2))
print(dividir(1, 0))
minimo, maximo = min_max([4, 9, 1, 7])
print(minimo, maximo)
print(saludar("Ana"))
```

SALIDA

```
5.0
None
1 9
Hola Ana
None
```

Ejemplo 2: return y finally

```
def prueba():
    try:
        return "desde try"
    finally:
        print("finally se ejecuta antes de retornar")

print(prueba())
```

SALIDA

```
finally se ejecuta antes de retornar
desde try
```

2.9 yield

Convierte una función en un **generador**. Cada `yield` entrega un valor y *suspende* la función conservando su estado; la ejecución se reanuda en la siguiente llamada a `next()`. `yield from` delega en otro iterable.

SINTAXIS

```
yield [expresión]
yield from iterable
```

PUNTOS CLAVE

- Los generadores producen valores bajo demanda y ahorran memoria.
- Cuando el generador termina se lanza `StopIteration` automáticamente.
- `yield` también es una expresión: `valor = yield x` permite recibir datos con `.send()`.
- `yield from sub` cede todos los valores de `sub` de forma eficiente.

EJEMPLOS

Ejemplo 1: Generador básico

```
def cuenta_regresiva(n):
    while n > 0:
        yield n
        n -= 1

for v in cuenta_regresiva(3):
    print(v)

gen = cuenta_regresiva(2)
print(next(gen))
print(next(gen))
try:
    next(gen)
except StopIteration:
    print("generador agotado")
```

SALIDA

```
3
2
1
2
1
generador agotado
```

Ejemplo 2: yield from

```
def concatenar(*iterables):
    for it in iterables:
        yield from it

print(list(concatenar([1, 2], "ab", range(3))))
```

SALIDA

```
[1, 2, 'a', 'b', 0, 1, 2]
```

Ejemplo 3: yield como expresión (send)

```
def acumulador():
    total = 0
    while True:
        valor = yield total
        total += valor

acc = acumulador()
next(acc)                # avanza hasta el primer yield
print(acc.send(10))
print(acc.send(5))
```

SALIDA

```
10
15
```

2.10 raise

Lanza una excepción. Sin argumentos, vuelve a lanzar la excepción que se está manejando. La cláusula `from` encadena explícitamente la causa original (`__cause__`), y `from None` suprime el contexto anterior.

SINTAXIS

```
raise [excepción [from causa]]
```

PUNTOS CLAVE

- Puede lanzarse una clase (`raise ValueError`) o una instancia (`raise ValueError('mensaje')`).
- Sólo se pueden lanzar objetos derivados de `BaseException` .
- Use excepciones específicas y mensajes claros.
- `raise` sin argumentos sólo es válido dentro de un bloque `except` .

EJEMPLOS

Ejemplo 1: Lanzar y capturar

```
def validar_edad(e):
    if e < 0:
        raise ValueError("La edad no puede ser negativa")
    return e

try:
    validar_edad(-5)
except ValueError as e:
    print("Capturado:", e)
```

SALIDA

```
Capturado: La edad no puede ser negativa
```

Ejemplo 2: Encadenar excepciones con from

```
try:
    try:
        int("abc")
    except ValueError as original:
        raise RuntimeError("Fallo al procesar el dato") from original
except RuntimeError as e:
    print(e)
    print("Causa:", type(e.__cause__).__name__)
```

SALIDA

```
Fallo al procesar el dato
Causa: ValueError
```

Ejemplo 3: Relanzar con raise

```
try:
    try:
        1 / 0
    except ZeroDivisionError:
        print("Registrando el error...")
        raise                  # vuelve a lanzar la misma excepción
except ZeroDivisionError:
    print("Capturada de nuevo en el nivel superior")
```

SALIDA

```
Registrando el error...
Capturada de nuevo en el nivel superior
```

2.11 break

Termina inmediatamente el bucle (`for` o `while`) más interno que lo contiene. Si el bucle tiene cláusula `else` , ésta **no** se ejecuta.

SINTAXIS

```
break
```

PUNTOS CLAVE

- Sólo es válida dentro de un bucle.
- Sólo afecta al bucle más interno; para salir de varios niveles use una función con `return` o una bandera.
- Es la base del patrón `while True: ... if cond: break`.

EJEMPLOS

Ejemplo 1: Buscar y salir

```
for n in [3, 7, 12, 5]:
    if n > 10:
        print("Primer valor mayor que 10:", n)
        break
else:
    print("Ningún valor mayor que 10")
```

SALIDA

```
Primer valor mayor que 10: 12
```

Ejemplo 2: Bucle infinito con salida controlada

```
claves = ["123", "abc", "secreta"]
i = 0
while True:
    intento = claves[i]
    i += 1
    if intento == "secreta":
        print("Acceso concedido tras", i, "intentos")
        break
    print("Clave incorrecta:", intento)
```

SALIDA

```
Clave incorrecta: 123
Clave incorrecta: abc
Acceso concedido tras 3 intentos
```

2.12 continue

Salta el resto del cuerpo del bucle en la iteración actual y pasa a la siguiente iteración.

SINTAXIS

```
continue
```

PUNTOS CLAVE

- Sólo es válida dentro de un bucle.
- Sirve para descartar casos y evitar anidamiento excesivo de `if`.
- En un `while`, vuelve a evaluar la condición; en un `for`, toma el siguiente elemento.

EJEMPLOS

Ejemplo 1: Saltar valores

```
for n in range(1, 10):
    if n % 2 == 0:
        continue          # ignora los pares
    print(n, end=" ")
print()
```

SALIDA

```
1 3 5 7 9
```

Ejemplo 2: continue en un while

```
i = 0
while i < 6:
    i += 1
    if i == 3:
        continue          # no imprime el 3
    print("i =", i)
```

SALIDA

```
i = 1
i = 2
i = 4
i = 5
i = 6
```

2.13 import

Carga un **módulo** (un archivo `.py` o paquete) y enlaza su nombre en el espacio de nombres actual. El módulo se ejecuta una sola vez y se guarda en caché (`sys.modules`).

SINTAXIS

```
import módulo [as alias][, módulo2 ...]
```

PUNTOS CLAVE

- `import a.b.c` importa el submódulo `c` del paquete `a.b` y se accede como `a.b.c` .
- `as` crea un alias: `import numpy as np` .
- Las importaciones se colocan al inicio del archivo (PEP 8): estándar, terceros, locales.
- Para importaciones dinámicas existe `importlib.import_module(nombre)` .

EJEMPLOS

Ejemplo 1: Importaciones básicas

```
import math
import os.path
import collections as col

print(math.sqrt(16))
print(os.path.basename("/tmp/archivo.txt"))
print(col.Counter("abracadabra"))
```

SALIDA

```
4.0
archivo.txt
Counter({'a': 5, 'b': 2, 'r': 2, 'c': 1, 'd': 1})
```

Ejemplo 2: Importación dinámica

```
import importlib

nombre = "json"
modulo = importlib.import_module(nombre)
print(modulo.dumps({"a": 1}))
```

SALIDA

```
{"a": 1}
```

2.14 from ... import

Importa **nombres concretos** de un módulo directamente al espacio de nombres actual, de modo que pueden usarse sin el prefijo del módulo.

SINTAXIS

```
from módulo import nombre [as alias][, ...]
from módulo import (a, b, c)
from . import hermano          # importación relativa
```

PUNTOS CLAVE

- `from módulo import *` importa todo lo público, pero **no se recomienda** porque contamina el espacio de nombres.
- Las importaciones relativas (`from . import x` , `from ..pkg import y`) sólo funcionan dentro de paquetes.
- `from __future__ import ...` activa características de versiones futuras y debe ir al inicio del archivo.

EJEMPLOS

Ejemplo 1: Importar nombres específicos

```
from math import pi, sqrt as raiz
from collections import (
    deque,
    defaultdict,
)
from datetime import date

print(round(pi, 4), raiz(81))
cola = deque([1, 2, 3])
cola.appendleft(0)
print(cola)
print(date(2026, 10, 18).strftime("%d/%m/%Y"))
```

SALIDA

```
3.1416 9.0
deque([0, 1, 2, 3])
18/10/2026
```

2.15 global

Declara que, dentro de una función, el nombre se refiere a la **variable global** del módulo. Sin esta declaración, una asignación dentro de la función crea una variable *local*.

SINTAXIS

```
global nombre[, nombre2 ...]
```

PUNTOS CLAVE

- Sólo es necesaria para **asignar** a una variable global; para leerla no hace falta.
- Para modificar objetos mutables globales (listas, diccionarios) mediante métodos tampoco es necesaria.
- El abuso de variables globales dificulta el mantenimiento del código.

EJEMPLOS

Ejemplo 1: global frente a variable local

```
contador = 0

def incrementar():
    global contador
    contador += 1

def sin_global():
    contador = 100       # crea una variable LOCAL; no afecta a la global

incrementar()
incrementar()
sin_global()
print(contador)
```

SALIDA

```
2
```

2.16 nonlocal

Dentro de una función **anidada**, indica que el nombre pertenece a la función contenedora más próxima (no es local ni global). Es la base de los *closures* con estado modificable.

SINTAXIS

```
nonlocal nombre[, nombre2 ...]
```

PUNTOS CLAVE

- La variable debe existir previamente en una función envolvente.
- Es el equivalente de `global` para ámbitos anidados.
- Útil para contadores, acumuladores y decoradores con estado.

EJEMPLOS

Ejemplo 1: Closure con contador

```
def crear_contador():
    cuenta = 0
    def incrementar():
        nonlocal cuenta
        cuenta += 1
        return cuenta
    return incrementar

c1 = crear_contador()
c2 = crear_contador()
print(c1(), c1(), c1())
print(c2())              # cada closure tiene su propio estado
```

SALIDA

```
1 2 3
1
```

2.17 assert

Comprueba que una condición sea verdadera; si es falsa lanza `AssertionError` (con el mensaje opcional). Se usa para **verificar supuestos internos** durante el desarrollo y las pruebas.

SINTAXIS

```
assert condición[, mensaje]
```

PUNTOS CLAVE

- Se desactiva al ejecutar Python con la opción `-O` (optimizado): no lo use para validar datos de entrada del usuario.
- Equivale a: `if __debug__ and not condición: raise AssertionError(mensaje)`.
- Evite poner una tupla como condición: `assert (x, 'msg')` siempre es verdadero.

EJEMPLOS

Ejemplo 1: Verificar precondiciones

```
def promedio(nums):
    assert len(nums) > 0, "La lista no puede estar vacía"
    return sum(nums) / len(nums)

print(promedio([2, 4, 6]))
try:
    promedio([])
except AssertionError as e:
    print("AssertionError:", e)
```

SALIDA

```
4.0
AssertionError: La lista no puede estar vacía
```

2.18 type

Disponible desde Python 3.12. Declara un **alias de tipo** de forma explícita y con evaluación diferida, con soporte para parámetros de tipo genéricos. Es una *soft keyword*.

SINTAXIS

```
type Nombre = tipo
type Nombre[T] = tipo_con_T
```

PUNTOS CLAVE

- El alias se evalúa de forma perezosa: puede referirse a nombres definidos más adelante.
- El valor del alias está disponible en `Alias.__value__`.
- Es sólo para anotaciones; no crea una clase nueva.

EJEMPLOS

Ejemplo 1: Alias de tipo

```
type Punto = tuple[float, float]
type Lista[T] = list[T]

def distancia(p: Punto) -> float:
    return (p[0] ** 2 + p[1] ** 2) ** 0.5

enteros: Lista[int] = [1, 2, 3]
print(distancia((3.0, 4.0)))
print(Punto.__value__)
print(enteros)
```

SALIDA

```
5.0
tuple[float, float]
[1, 2, 3]
```

CAPÍTULO 3

3. Sentencias de control de flujo

Estas sentencias compuestas determinan qué código se ejecuta y cuántas veces: condicionales, bucles y concordancia de patrones.

3.1 if / elif / else

Ejecuta un bloque **sólo si** su condición es verdadera. Se evalúan las condiciones en orden y se ejecuta únicamente el primer bloque cuya condición se cumpla; `else` se ejecuta si ninguna anterior se cumplió.

SINTAXIS

```
if condición1:
    bloque1
elif condición2:
    bloque2
else:
    bloque_por_defecto
```

PUNTOS CLAVE

- `elif` y `else` son opcionales; puede haber cualquier cantidad de `elif`.
- Cualquier objeto puede usarse como condición: son *falsos* `False`, `None`, `0`, `0.0`, `''`, `[]`, `{}`, `()`, `set()`; todo lo demás es *verdadero*.
- Se pueden combinar condiciones con `and`, `or`, `not` y encadenar comparaciones (`0 < x < 10`).
- Existe también la **expresión condicional** `a if cond else b`.

EJEMPLOS

Ejemplo 1: Cadena de condiciones

```
nota = 85

if nota >= 90:
    print("Sobresaliente")
elif nota >= 70:
    print("Aprobado")
elif nota >= 50:
    print("Suficiente")
else:
    print("Reprobado")
```

SALIDA

```
Aprobado
```

Ejemplo 2: Veracidad de los objetos

```
for valor in [0, 1, "", "hola", [], [0], None]:
    if valor:
        print(repr(valor), "-> verdadero")
    else:
        print(repr(valor), "-> falso")
```

SALIDA

```
0 -> falso
1 -> verdadero
'' -> falso
'hola' -> verdadero
[] -> falso
[0] -> verdadero
None -> falso
```

Ejemplo 3: Comparaciones encadenadas y expresión condicional

```
x = 7
if 0 < x < 10 and x % 2 == 1:
    print("x está entre 0 y 10 y es impar")

edad = 20
estado = "adulto" if edad >= 18 else "menor"
print(estado)
```

SALIDA

```
x está entre 0 y 10 y es impar
adulto
```

3.2 while

Repite un bloque **mientras** la condición sea verdadera. La condición se evalúa antes de cada iteración. La cláusula opcional `else` se ejecuta cuando la condición se vuelve falsa, pero **no** si el bucle termina con `break`.

SINTAXIS

```
while condición:
    bloque
[else:
    bloque_else]
```

PUNTOS CLAVE

- Asegúrese de que algo dentro del bucle cambie la condición, o tendrá un bucle infinito.
- `while True:` con `break` es un patrón habitual para repetir hasta que ocurra un evento.
- `break` termina el bucle y `continue` salta a la siguiente iteración.

EJEMPLOS

Ejemplo 1: Bucle con contador

```
n = 1
while n <= 5:
    print(n, end=" ")
    n += 1
print()
```

SALIDA

```
1 2 3 4 5
```

Ejemplo 2: while con else

```
intentos = 0
while intentos < 3:
    intentos += 1
    print("Intento", intentos)
else:
    print("La condición se volvió falsa: se ejecuta else")

pila = [1, 2, 3, 99, 4]
while pila:
    v = pila.pop()
    if v == 99:
        print("Encontrado 99, se ejecuta break")
        break
else:
    print("No se ve este mensaje")
```

SALIDA

```
Intento 1
Intento 2
Intento 3
La condición se volvió falsa: se ejecuta else
Encontrado 99, se ejecuta break
```

Ejemplo 3: Algoritmo de Euclides (máximo común divisor)

```
a, b = 48, 18
while b != 0:
    a, b = b, a % b
print("MCD:", a)
```

SALIDA

```
MCD: 6
```

3.3 for

Recorre los elementos de cualquier objeto iterable (listas, cadenas, diccionarios, rangos, archivos, generadores...). En cada vuelta el elemento actual se asigna al objetivo. Como en `while`, la cláusula `else` se ejecuta si el bucle termina sin `break`.

SINTAXIS

```
for objetivo in iterable:
    bloque
[else:
    bloque_else]
```

PUNTOS CLAVE

- `range(inicio, fin, paso)` genera secuencias numéricas (el fin es excluyente).
- `enumerate(iterable, start=0)` entrega pares (índice, valor).
- `zip(a, b)` recorre varias secuencias en paralelo.
- El objetivo puede ser una tupla: `for clave, valor in dic.items():`.
- No modifique la colección que está recorriendo; itere sobre una copia.

EJEMPLOS

Ejemplo 1: Recorrer secuencias y rangos

```python
for fruta in ["manzana", "pera", "uva"]:
    print(fruta)

for i in range(3):
    print("i =", i)

for i in range(2, 12, 3):           # inicio, fin (excluido), paso
    print(i, end=" ")
print()

for letra in "Hola":
    print(letra, end="-")
print()
```

SALIDA

```
manzana
pera
uva
i = 0
i = 1
i = 2
2 5 8 11
H-o-l-a-
```

Ejemplo 2: Diccionarios, enumerate y zip

```python
precios = {"pan": 1.5, "leche": 2.0}
for producto, precio in precios.items():
    print(f"{producto}: {precio}")

for pos, color in enumerate(["rojo", "verde", "azul"], start=1):
    print(pos, color)

nombres = ["Ana", "Luis"]
edades = [30, 25]
for nombre, edad in zip(nombres, edades):
    print(nombre, "tiene", edad)
```

SALIDA

```
pan: 1.5
leche: 2.0
1 rojo
2 verde
3 azul
Ana tiene 30
Luis tiene 25
```

Ejemplo 3: for con else: números primos

```python
for n in range(2, 20):
    for d in range(2, n):
        if n % d == 0:
            break               # no es primo
    else:
        print(n, end=" ")       # no hubo break: es primo
print()
```

SALIDA

```
2 3 5 7 11 13 17 19
```

3.4 match / case

Disponible desde Python 3.10. Realiza **coincidencia estructural de patrones** (*structural pattern matching*). Es mucho más potente que un `switch` : puede comparar literales, descomponer secuencias y diccionarios, verificar tipos y extraer valores a variables.

SINTAXIS

```
match expresión:
    case patrón1 [if guarda]:
        bloque1
    case patrón2 | patrón3:
        bloque2
    case _:
        bloque_por_defecto
```

PUNTOS CLAVE

- **Literales**: `case 200:` , `case 'rojo':` , `case None:` .
- **Captura**: `case x:` asigna el valor a `x` (un nombre sin punto siempre captura).
- **Comodín**: `case _:` coincide con cualquier cosa y no enlaza nada.
- **Alternativas**: `case 301 | 302:` .
- **Secuencias**: `case [x, y]` , `case [primero, *resto]` .
- **Mapeos**: `case {'tipo': 'circulo', 'radio': r}` .
- **Clases**: `case Punto(x=0, y=y)` .
- **Guardas**: `case n if n > 10:` añade una condición adicional.
- `as` : `case str() as texto:` captura el valor completo.

EJEMPLOS

Ejemplo 1: Literales y alternativas

```
def estado_http(codigo):
    match codigo:
        case 200:
            return "OK"
        case 301 | 302:
            return "Redirección"
        case 404:
            return "No encontrado"
        case _:
            return "Código desconocido"

for c in (200, 302, 404, 500):
    print(c, "->", estado_http(c))
```

SALIDA

```
200 -> OK
302 -> Redirección
404 -> No encontrado
500 -> Código desconocido
```

Ejemplo 2: Patrones de secuencia

```python
def describir(dato):
    match dato:
        case []:
            return "lista vacía"
        case [x]:
            return f"un elemento: {x}"
        case [x, y]:
            return f"dos elementos: {x} y {y}"
        case [x, *resto]:
            return f"empieza con {x} y tiene {len(resto)} más"
        case _:
            return "no es una lista"

print(describir([]))
print(describir([5]))
print(describir([1, 2]))
print(describir([1, 2, 3, 4]))
print(describir(42))
```

SALIDA

```
lista vacía
un elemento: 5
dos elementos: 1 y 2
empieza con 1 y tiene 3 más
no es una lista
```

Ejemplo 3: Clases, diccionarios y guardas

```python
from dataclasses import dataclass

@dataclass
class Punto:
    x: int
    y: int

def analizar(obj):
    match obj:
        case Punto(x=0, y=0):
            return "origen"
        case Punto(x=0, y=y):
            return f"sobre el eje Y en {y}"
        case Punto(x=x, y=0):
            return f"sobre el eje X en {x}"
        case Punto(x=x, y=y) if x == y:
            return f"sobre la diagonal en {x}"
        case {"tipo": "circulo", "radio": r}:
            return f"círculo de radio {r}"
        case str() as texto:
            return f"texto: {texto}"
        case _:
            return "otro objeto"

for dato in [Punto(0, 0), Punto(0, 4), Punto(3, 0), Punto(2, 2),
             {"tipo": "circulo", "radio": 5}, "hola", 3.14]:
    print(analizar(dato))
```

SALIDA

```
origen
sobre el eje Y en 4
sobre el eje X en 3
sobre la diagonal en 2
círculo de radio 5
texto: hola
otro objeto
```

CAPÍTULO 4

4. Manejo de excepciones y recursos

Estas sentencias permiten reaccionar ante errores en tiempo de ejecución y garantizar que los recursos (archivos, conexiones, bloqueos) se liberen correctamente.

4.1 try / except / else / finally

Intenta ejecutar un bloque y **captura las excepciones** que ocurran. Cada cláusula tiene un papel preciso.

SINTAXIS

```
try:
    bloque_riesgoso
except TipoError [as e]:
    manejo_del_error
else:
    si_no_hubo_error
finally:
    siempre_se_ejecuta
```

PUNTOS CLAVE

- `try` : código que puede fallar.
- `except` : se ejecuta si se lanza la excepción indicada. Puede haber varias, de la más específica a la más general.
- `else` : se ejecuta sólo si el `try` terminó **sin** excepciones.
- `finally` : se ejecuta **siempre**, ocurra o no una excepción (ideal para limpieza).
- `except (A, B) as e:` captura varios tipos a la vez.
- Evite `except:` desnudo o `except Exception: pass` , porque oculta errores reales.

EJEMPLOS

Ejemplo 1: Las cuatro cláusulas

```python
def dividir(a, b):
    try:
        resultado = a / b
    except ZeroDivisionError:
        print("No se puede dividir entre cero")
        return None
    except TypeError as e:
        print("Tipos inválidos:", e)
        return None
    else:
        print("División exitosa")
        return resultado
    finally:
        print("Fin del intento")

print(dividir(10, 2))
print(dividir(1, 0))
print(dividir(1, "x"))
```

SALIDA

```
División exitosa
Fin del intento
5.0
No se puede dividir entre cero
Fin del intento
None
Tipos inválidos: unsupported operand type(s) for /: 'int' and 'str'
Fin del intento
None
```

Ejemplo 2: Varias excepciones y jerarquía

```python
datos = ["10", "abc", None, "0"]

for d in datos:
    try:
        print(100 // int(d))
    except (ValueError, TypeError) as e:
        print(f"Dato inválido {d!r}: {type(e).__name__}")
    except ZeroDivisionError:
        print("División entre cero")
```

SALIDA

```
10
Dato inválido 'abc': ValueError
Dato inválido None: TypeError
División entre cero
```

Ejemplo 3: Excepciones personalizadas

```
class SaldoInsuficiente(Exception):
    def __init__(self, saldo, monto):
        super().__init__(f"Saldo {saldo} insuficiente para retirar {monto}")
        self.saldo = saldo
        self.monto = monto

def retirar(saldo, monto):
    if monto > saldo:
        raise SaldoInsuficiente(saldo, monto)
    return saldo - monto

try:
    retirar(100, 250)
except SaldoInsuficiente as e:
    print("Error:", e)
    print("Faltan:", e.monto - e.saldo)
```

SALIDA

```
Error: Saldo 100 insuficiente para retirar 250
Faltan: 150
```

4.2 except* (grupos de excepciones)

Disponible desde Python 3.11. Permite manejar un `ExceptionGroup` (varias excepciones lanzadas a la vez, típico en programación concurrente). Cada cláusula `except*` procesa el subconjunto de excepciones que coinciden con su tipo.

SINTAXIS

```
try:
    bloque
except* TipoA as grupo:
    manejo_A
except* TipoB as grupo:
    manejo_B
```

PUNTOS CLAVE

- No se puede mezclar `except` y `except*` en un mismo `try`.
- Las cláusulas `except*` pueden ejecutarse varias veces para un mismo grupo.
- Las excepciones no manejadas se relanzan en un nuevo grupo.

EJEMPLOS

Ejemplo 1: Manejar un ExceptionGroup

```
def tareas():
    raise ExceptionGroup("errores múltiples", [
        ValueError("valor malo"),
        TypeError("tipo malo"),
        ValueError("otro valor malo"),
    ])

try:
    tareas()
except* ValueError as grupo:
    print("ValueError:", [str(e) for e in grupo.exceptions])
except* TypeError as grupo:
    print("TypeError:", [str(e) for e in grupo.exceptions])
```

SALIDA

```
ValueError: ['valor malo', 'otro valor malo']
TypeError: ['tipo malo']
```

4.3 with

Ejecuta un bloque dentro de un **gestor de contexto** (*context manager*). Garantiza que se llame a `__enter__()` al entrar y a `__exit__()` al salir, **incluso si ocurre una excepción**. Es la forma segura y limpia de manejar archivos, conexiones, bloqueos y transacciones.

SINTAXIS

```
with expresión [as nombre][, expresión2 [as nombre2]]:
    bloque
```

PUNTOS CLAVE

- Reemplaza el patrón `try/finally` para liberar recursos.
- Puede abrir varios recursos a la vez, separados por comas o entre paréntesis (3.10+).
- Se pueden crear gestores propios con una clase (`__enter__` / `__exit__`) o con `@contextlib.contextmanager` .
- Si `__exit__` devuelve `True` , la excepción se suprime.

EJEMPLOS

Ejemplo 1: Archivos con with

```
with open("/tmp/ejemplo.txt", "w", encoding="utf-8") as f:
    f.write("Primera línea\n")
    f.write("Segunda línea\n")

with open("/tmp/ejemplo.txt", encoding="utf-8") as f:
    for linea in f:
        print(linea.rstrip())

print("¿Archivo cerrado?", f.closed)
```

SALIDA

```
Primera línea
Segunda línea
¿Archivo cerrado? True
```

Ejemplo 2: Varios gestores a la vez

```
with (
    open("/tmp/ejemplo.txt", encoding="utf-8") as origen,
    open("/tmp/copia.txt", "w", encoding="utf-8") as destino,
):
    for linea in origen:
        destino.write(linea.upper())

print(open("/tmp/copia.txt", encoding="utf-8").read())
```

SALIDA

```
PRIMERA LÍNEA
SEGUNDA LÍNEA
```

Ejemplo 3: Gestor de contexto como clase

```
class Temporizador:
    def __enter__(self):
        print("Entrando al bloque")
        return self

    def __exit__(self, tipo, valor, traza):
        print("Saliendo; excepción:", tipo.__name__ if tipo else None)
        return False          # False: no suprime la excepción

with Temporizador() as t:
    print("Dentro del bloque")

try:
    with Temporizador():
        raise ValueError("fallo")
except ValueError:
    print("La excepción se propagó")
```

SALIDA

```
Entrando al bloque
Dentro del bloque
Saliendo; excepción: None
Entrando al bloque
Saliendo; excepción: ValueError
La excepción se propagó
```

Ejemplo 4: Gestor con @contextmanager

```
from contextlib import contextmanager

@contextmanager
def etiqueta(nombre):
    print(f"<{nombre}>")
    try:
        yield
    finally:
        print(f"</{nombre}>")

with etiqueta("seccion"):
    print("contenido")
```

SALIDA

```
<seccion>
contenido
</seccion>
```

CAPÍTULO 5

5. Sentencias de definición

Permiten crear funciones, clases y rutinas asíncronas. Al ejecutarse, estas sentencias *vinculan un nombre* a un nuevo objeto (función o clase).

5.1 def (definición de funciones)

Define una **función**. Crea un objeto función y lo asocia al nombre. El cuerpo no se ejecuta hasta que la función es llamada.

SINTAXIS

```
def nombre(parámetros) [-> tipo]:
    """docstring"""
    cuerpo
```

PUNTOS CLAVE

- **Parámetros posicionales o por nombre**, con valores por defecto (`b=2`).
- `*args` recoge argumentos posicionales extra en una tupla; `**kwargs` recoge los de palabra clave en un diccionario.
- **Sólo posicionales**: parámetros antes de `/` . **Sólo por nombre**: parámetros después de `*` o `*args` .
- Los valores por defecto se evalúan **una vez**, al definir la función: evite objetos mutables como `[]` .
- La primera cadena del cuerpo es el *docstring* (se obtiene con `help()` o `__doc__`).
- Las funciones son objetos: pueden asignarse, pasarse como argumento y devolverse.

EJEMPLOS

Ejemplo 1: Tipos de parámetros

```
def ejemplo(a, b=2, /, c=3, *args, d, e=5, **kwargs):
    print(f"a={a} b={b} c={c} args={args} d={d} e={e} kwargs={kwargs}")

ejemplo(1, d=4)
ejemplo(1, 20, 30, 40, 50, d=4, extra="x")
```

SALIDA

```
a=1 b=2 c=3 args=() d=4 e=5 kwargs={}
a=1 b=20 c=30 args=(40, 50) d=4 e=5 kwargs={'extra': 'x'}
```

Ejemplo 2: Docstring, anotaciones y recursión

```
def factorial(n: int) -> int:
    """Devuelve el factorial de n (n >= 0)."""
    if n <= 1:
        return 1
    return n * factorial(n - 1)

print(factorial(5))
print(factorial.__doc__)
```

SALIDA

```
120
Devuelve el factorial de n (n >= 0).
```

Ejemplo 3: Trampa del argumento mutable por defecto

```
def mal(x, lista=[]):
    lista.append(x)
    return lista

def bien(x, lista=None):
    if lista is None:
        lista = []
    lista.append(x)
    return lista

print(mal(1), mal(2))      # la misma lista se reutiliza
print(bien(1), bien(2))    # cada llamada crea una lista nueva
```

SALIDA

```
[1, 2] [1, 2]
[1] [2]
```

Ejemplo 4: Funciones como objetos

```
def doble(x):
    return x * 2

def aplicar(func, valores):
    return [func(v) for v in valores]

operacion = doble              # asignar una función a otro nombre
print(operacion(21))
print(aplicar(doble, [1, 2, 3]))
print(aplicar(str.upper, ["a", "b"]))
```

SALIDA

```
42
[2, 4, 6]
['A', 'B']
```

5.2 class (definición de clases)

Define una **clase**, es decir, una plantilla para crear objetos. El cuerpo se ejecuta al definirse y su espacio de nombres se convierte en los atributos de la clase.

SINTAXIS

```
class Nombre[(Base1, Base2, ...)]:
    atributos_de_clase
    def método(self, ...):
        ...
```

PUNTOS CLAVE

- `__init__(self, ...)` es el inicializador de cada instancia.
- El primer parámetro de los métodos (`self`) es la instancia.
- **Herencia**: `class Perro(Animal)` ; use `super()` para llamar a la clase base.
- `@classmethod` recibe la clase (`cls`); `@staticmethod` no recibe ni clase ni instancia; `@property` expone un método como atributo.
- Los métodos especiales (`__str__` , `__repr__` , `__eq__` , `__len__` ...) personalizan el comportamiento.

EJEMPLOS

Ejemplo 1: Clase con herencia y polimorfismo

```python
class Animal:
    especie = "desconocida"            # atributo de clase

    def __init__(self, nombre):
        self.nombre = nombre           # atributo de instancia

    def hablar(self):
        return f"{self.nombre} hace un sonido"

    def __str__(self):
        return f"{self.especie}: {self.nombre}"

class Perro(Animal):
    especie = "canino"

    def hablar(self):
        return f"{self.nombre} dice: ¡Guau!"

class Gato(Animal):
    especie = "felino"

    def hablar(self):
        base = super().hablar()
        return base + " (miau)"

for animal in [Perro("Rex"), Gato("Misu"), Animal("Bicho")]:
    print(animal, "|", animal.hablar())
```

SALIDA

```
canino: Rex | Rex dice: ¡Guau!
felino: Misu | Misu hace un sonido (miau)
desconocida: Bicho | Bicho hace un sonido
```

Ejemplo 2: property, classmethod y staticmethod

```python
class Circulo:
    PI = 3.14159

    def __init__(self, radio):
        self._radio = radio

    @property
    def radio(self):
        return self._radio

    @radio.setter
    def radio(self, valor):
        if valor <= 0:
            raise ValueError("El radio debe ser positivo")
        self._radio = valor

    @property
    def area(self):
        return Circulo.PI * self._radio ** 2

    @classmethod
    def desde_diametro(cls, diametro):
        return cls(diametro / 2)

    @staticmethod
    def es_valido(radio):
        return radio > 0

c = Circulo.desde_diametro(10)
print(c.radio, round(c.area, 2))
c.radio = 3
print(round(c.area, 2), Circulo.es_valido(-1))
```

SALIDA

```
5.0 78.54
28.27 False
```

Ejemplo 3: Métodos especiales

```python
class Vector:
    def __init__(self, x, y):
        self.x, self.y = x, y

    def __add__(self, otro):
        return Vector(self.x + otro.x, self.y + otro.y)

    def __eq__(self, otro):
        return (self.x, self.y) == (otro.x, otro.y)

    def __repr__(self):
        return f"Vector({self.x}, {self.y})"

    def __len__(self):
        return 2

v = Vector(1, 2) + Vector(3, 4)
print(v, v == Vector(4, 6), len(v))
```

SALIDA

```
Vector(4, 6) True 2
```

5.3 Decoradores (@)

Un **decorador** es un invocable que recibe una función (o clase) y devuelve otra versión modificada. La sintaxis `@dec` antes de un `def` o `class` equivale a `función = dec(función)`.

SINTAXIS

```
@decorador
def función(): ...

@decorador(argumentos)
class Clase: ...
```

PUNTOS CLAVE

- Sirven para añadir comportamiento (registro, caché, validación, medición de tiempo) sin tocar la función original.
- Use `functools.wraps` dentro del decorador para conservar el nombre y el docstring.
- Pueden recibir argumentos, lo que requiere un nivel extra de anidamiento.
- Se pueden apilar varios decoradores: se aplican de abajo hacia arriba.

EJEMPLOS

Ejemplo 1: Decorador simple

```
import functools

def registrar(func):
    @functools.wraps(func)
    def envoltura(*args, **kwargs):
        print(f"Llamando a {func.__name__}{args}")
        resultado = func(*args, **kwargs)
        print(f"Resultado: {resultado}")
        return resultado
    return envoltura

@registrar
def sumar(a, b):
    return a + b

sumar(3, 4)
print(sumar.__name__)
```

SALIDA

```
Llamando a sumar(3, 4)
Resultado: 7
sumar
```

Ejemplo 2: Decorador con argumentos y apilado

```
import functools

def repetir(veces):
    def decorador(func):
        @functools.wraps(func)
        def envoltura(*args, **kwargs):
            for _ in range(veces):
                resultado = func(*args, **kwargs)
            return resultado
        return envoltura
    return decorador

def mayusculas(func):
    @functools.wraps(func)
    def envoltura(*a, **k):
        return func(*a, **k).upper()
    return envoltura

@repetir(2)
@mayusculas
def saludo(nombre):
    texto = f"hola {nombre}"
    print(texto)
    return texto

saludo("ana")
```

SALIDA

```
hola ana
hola ana
```

Ejemplo 3: Decoradores de la biblioteca estándar

```
from functools import lru_cache

@lru_cache(maxsize=None)
def fibonacci(n):
    return n if n < 2 else fibonacci(n - 1) + fibonacci(n - 2)

print(fibonacci(50))
print(fibonacci.cache_info())
```

SALIDA

```
12586269025
CacheInfo(hits=48, misses=51, maxsize=None, currsize=51)
```

5.4 async def / await / async for / async with

Soportan la **programación asíncrona** con `asyncio`. `async def` define una *corrutina*; `await` suspende la corrutina hasta que otra operación asíncrona termine, cediendo el control para que se ejecuten otras tareas.

SINTAXIS

```
async def nombre(...):
    resultado = await corrutina
    async for x in iterable_asincrono: ...
    async with gestor_asincrono as g: ...
```

PUNTOS CLAVE

- Llamar a una función `async def` no la ejecuta: devuelve un objeto corrutina.
- `await` sólo puede usarse dentro de `async def`.

- `asyncio.run(main())` inicia el bucle de eventos; `asyncio.gather(...)` ejecuta tareas de forma concurrente.
- `async for` recorre iteradores asíncronos; `async with` usa gestores de contexto asíncronos (`__aenter__` / `__aexit__`).
- Ideal para tareas limitadas por E/S (red, archivos, bases de datos), no para cálculo intensivo.

EJEMPLOS

Ejemplo 1: Corrutinas concurrentes

```python
import asyncio

async def tarea(nombre, espera):
    await asyncio.sleep(espera)
    print(f"{nombre} terminó tras {espera}s")
    return nombre.upper()

async def main():
    resultados = await asyncio.gather(
        tarea("lenta", 0.3),
        tarea("rápida", 0.1),
    )
    print(resultados)

asyncio.run(main())
```

SALIDA

```
rápida terminó tras 0.1s
lenta terminó tras 0.3s
['LENTA', 'RÁPIDA']
```

Ejemplo 2: async for y async with

```python
import asyncio

async def contar(n):
    for i in range(n):
        await asyncio.sleep(0.01)
        yield i                      # generador asíncrono

class Recurso:
    async def __aenter__(self):
        print("abrir recurso")
        return self

    async def __aexit__(self, tipo, valor, traza):
        print("cerrar recurso")

async def main():
    async with Recurso():
        async for v in contar(3):
            print("valor", v)

asyncio.run(main())
```

SALIDA

```
abrir recurso
valor 0
valor 1
valor 2
cerrar recurso
```

CAPÍTULO 6

6. Construcciones que parecen sentencias

Algunas construcciones se confunden con sentencias, pero en realidad son **expresiones**: producen un valor y pueden aparecer dentro de otras expresiones.

6.1 lambda, comprensiones y expresión condicional

Son expresiones: se evalúan a un valor. No pueden contener sentencias (una `lambda` sólo admite una expresión, no `return`, `if` en bloque ni asignaciones).

SINTAXIS

```
lambda parámetros: expresión
[expr for x in iterable if cond]
valor_si if condición else valor_no
```

PUNTOS CLAVE

- `lambda`: función anónima de una sola expresión.
- **Comprensiones**: de lista `[]`, de conjunto `{}`, de diccionario `{k: v}` y generadores `()`.
- **Expresión condicional**: `a if cond else b`.
- `await`, `yield` y `:=` también son expresiones.

EJEMPLOS

Ejemplo 1: Ejemplos

```python
cuadrado = lambda x: x ** 2
print(cuadrado(6))

nombres = ["luis", "ana", "carlos"]
print(sorted(nombres, key=lambda s: len(s)))

pares = [n for n in range(10) if n % 2 == 0]
cuadrados = {n: n ** 2 for n in range(4)}
unicos = {len(s) for s in nombres}
suma = sum(n * n for n in range(5))      # expresión generadora
print(pares, cuadrados, unicos, suma)

print("par" if 8 % 2 == 0 else "impar")
```

SALIDA

```
36
['ana', 'luis', 'carlos']
[0, 2, 4, 6, 8] {0: 0, 1: 1, 2: 4, 3: 9} {3, 4, 6} 30
par
```

CAPÍTULO 7

7. Ejemplo integrador

El siguiente programa combina muchas de las sentencias vistas: `import`, `class`, `def`, `assert`, `try/except/else`, `raise ... from`, `yield from`, `for`, `if`, `match/case`, `continue`, f-strings y asignaciones.

```
from dataclasses import dataclass

class StockError(Exception):
    """Error de inventario."""

@dataclass
class Producto:
    nombre: str
    precio: float
    stock: int = 0

class Inventario:
    def __init__(self):
        self._items = {}

    def agregar(self, producto):
        assert producto.precio > 0, "Precio inválido"
        self._items[producto.nombre] = producto

    def vender(self, nombre, cantidad):
        try:
            p = self._items[nombre]
        except KeyError:
            raise StockError(f"No existe el producto: {nombre}") from None
        if cantidad > p.stock:
            raise StockError(f"Stock insuficiente de {nombre}")
        p.stock -= cantidad
        return p.precio * cantidad

    def __iter__(self):
        yield from self._items.values()

inv = Inventario()
for nombre, precio, stock in [("Pan", 1.5, 10), ("Leche", 2.0, 5), ("Queso", 8.0, 2)]:
    inv.agregar(Producto(nombre, precio, stock))

total = 0
for nombre, cantidad in [("Pan", 4), ("Leche", 9), ("Vino", 1), ("Queso", 2)]:
    try:
        total += inv.vender(nombre, cantidad)
    except StockError as e:
        print("Error:", e)
        continue
    else:
        print(f"Vendido: {cantidad} x {nombre}")

print(f"Total vendido: {total}")

for p in inv:
    match p:
        case Producto(stock=0):
            print(p.nombre, "-> agotado")
        case Producto(stock=s) if s < 5:
            print(p.nombre, "-> stock bajo:", s)
        case _:
            print(p.nombre, "-> stock normal:", p.stock)
```

SALIDA

```
Vendido: 4 x Pan
Error: Stock insuficiente de Leche
Error: No existe el producto: Vino
Vendido: 2 x Queso
Total vendido: 22.0
Pan -> stock normal: 6
Leche -> stock normal: 5
Queso -> agotado
```

CAPÍTULO 8

8. Tabla resumen de sentencias

Vista rápida de todas las sentencias tratadas en esta guía, su clasificación, su propósito y la versión de Python en la que están disponibles.

Sentencia	Tipo	Propósito	Versión
`Expresión`	Simple	Evaluar una expresión (llamadas a funciones)	Todas
`Asignación =`	Simple	Vincular nombres a objetos	Todas
`Asignación aumentada +=`	Simple	Operar y asignar a la vez	2.0+
`Asignación anotada x: T`	Simple	Declarar el tipo de una variable	3.6+
`Operador morsa :=`	Expresión	Asignar dentro de una expresión	3.8+
`pass`	Simple	No hacer nada (marcador de posición)	Todas
`del`	Simple	Eliminar referencias	Todas
`return`	Simple	Devolver un valor desde una función	Todas
`yield / yield from`	Simple	Crear generadores	2.2+ / 3.3+
`raise`	Simple	Lanzar una excepción	Todas
`break`	Simple	Salir de un bucle	Todas
`continue`	Simple	Pasar a la siguiente iteración	Todas
`import / from ... import`	Simple	Cargar módulos y nombres	Todas
`global`	Simple	Usar una variable global en una función	Todas
`nonlocal`	Simple	Usar una variable de una función envolvente	3.0+
`assert`	Simple	Verificar supuestos internos	Todas
`type`	Simple	Declarar alias de tipo	3.12+
`if / elif / else`	Compuesta	Ejecución condicional	Todas
`while`	Compuesta	Repetir mientras se cumpla una condición	Todas
`for`	Compuesta	Recorrer un iterable	Todas
`match / case`	Compuesta	Coincidencia estructural de patrones	3.10+
`try / except / else / finally`	Compuesta	Manejar excepciones	Todas
`except*`	Compuesta	Manejar grupos de excepciones	3.11+
`with`	Compuesta	Gestionar recursos con contexto	2.5+
`def`	Compuesta	Definir funciones	Todas
`class`	Compuesta	Definir clases	Todas
`@decorador`	Compuesta	Modificar funciones y clases	2.4+
`async def / await`	Compuesta	Definir y esperar corrutinas	3.5+
`async for / async with`	Compuesta	Iteración y contexto asíncronos	3.5+

Nota: el operador morsa (`:=`) y las construcciones del último capítulo son *expresiones*; se incluyen por su cercanía con las sentencias.